

Назва проекту: **DynamicWallpaper2**

Виконавець: Фурдин Павло

Група: 3-4

Інтеграція з WorkerW

Використання Win32 API для вбудовування вікна рендерингу під іконки робочого столу. Процес не залежить від графічного прискорювача і реалізується через маніпуляцію ієрархією вікон Windows. Відправка системного повідомлення створює новий екземпляр WorkerW, після чого SetParent закріплює вікно програми як дочірнє.

```
import ctypes
import win32gui
import win32con

def embed_to_desktop(hwnd):
    progman = win32gui.FindWindowW("Progman", None)
    # Генерація WorkerW вікна
    win32gui.SendMessageTimeoutW(
        progman, 0x052C, 0, 0, win32con.SMT0_NORMAL, 1000
    )

    workerw = []
    def enum_windows(wnd, param):
        def_view = win32gui.FindWindowExW(wnd, 0, "SHELLDLL_DefView", None)
        if def_view:
            workerw.append(win32gui.FindWindowExW(0, wnd, "WorkerW", None))
        return True

    win32gui.EnumWindows(enum_windows, 0)
    if workerw:
        ctypes.windll.user32.SetParent(hwnd, workerw[0])
```

Типи рендерингу ефектів

Архітектура підтримує два незалежні механізми відмальовування:

1. **CPU-рендеринг:** Використовує **QPainter** для базових двовимірних генерацій (оптимально для інтегрованої графіки).
2. **GPU-рендеринг:** Використовує OpenGL для складних шейдерних ефектів.

Шлях виконання (Render Path) є інкапсульованою властивістю самого класу ефекту. Ядро програми динамічно адаптується до типу ефекту, звертаючись до його константи `RENDER_TYPE`.

```
class BaseEffect:
    RENDER_TYPE = "CPU" # або "GPU"

    def render(self, context):
        raise NotImplementedError

class ShaderEffect(BaseEffect):
    RENDER_TYPE = "GPU"

    def render(self, gl_context):
        gl_context.glUseProgram(self.shader)
        # OpenGL render logic...

class ParticleEffect(BaseEffect):
    RENDER_TYPE = "CPU"

    def render(self, qpainter):
        qpainter.drawEllipse(...)
```


Система плагінів та ООП

Реалізовано модульну архітектуру на основі принципів ООП. Кожен візуальний плагін успадковується від абстрактного класу BaseEffect, реалізуючи поліморфний метод update() та render().

Для забезпечення безперервної роботи впроваджено механізм Hot-Reload. Використання модуля importlib дозволяє динамічно перекомпілювати та імпортувати класи ефектів у пам'ять під час виконання програми (Runtime).

```
import importlib
import sys

def load_effect_plugin(module_name, class_name):
    if module_name in sys.modules:
        module = importlib.reload(
            sys.modules[module_name])
    else:
        module = importlib.import_module(
            f"plugins.{module_name}")

    effect_class = getattr(module, class_name)
    if isinstance(effect_class, BaseEffect):
        return effect_class()
    raise TypeError("Plugin must inherit from
BaseEffect")
```

Багатопотоковість (QThread)

Запобігання блокуванню головного циклу (UI Thread) реалізовано через делегування ресурсомістких завдань у дочірні потоки за допомогою QThread.

Логіка рендерингу відео та захоплення аудіо (через PyAudio для FFT-аналізу) виконується асинхронно.

Комунікація між обчислювальними потоками та графічним інтерфейсом відбувається виключно через потокобезпечну систему сигналів та слотів (Signals/Slots).

```
from PyQt6.QtCore import QThread, pyqtSignal
import pyaudio

class AudioCaptureThread(QThread):
    fft_data_ready = pyqtSignal(list)

    def run(self):
        p = pyaudio.PyAudio()
        stream = p.open(format=pyaudio.paInt16,
                        channels=1,
                        rate=44100,
                        input=True,
                        frames_per_buffer=1024)
        while not self.isInterruptRequested():
            data = stream.read(1024)
            fft_result = self.process_fft(data)
            self.fft_data_ready.emit(fft_result)
```